

Project Plan

Ahmed Fawaizat Oyindamola (fawah595)
Abdul Gaffar Abubakar Passum (abuab954)
Varun Gurupurandar (vargu125)
Mehran Mamivand (mehma172)

February 11, 2026

Version 1.0



Status

Reviewed	Firstname Lastname	date
Approved	Firstname Lastname	date

Project Identity

Group E-mail: tsks23-group3@liu.se

Orderer & Customer: Danyo Danev
Phone: +46 (0)13 28 13 35
E-mail: danyo.danev@liu.se

Supervisor: Oksana Moryakova
Phone: +46 (0)13-28 15 53
E-mail: oksana.moryakova@liu.se

Supervisor: David Nordlund
Phone: +46 (0)13-28 19 53
E-mail: david.nordlund@liu.se

Supervisor: Henrik Åkesson
Phone: +46 (0)13-28 28 95
E-mail: henrik.akesson@liu.se

Course Responsible: Danyo Danev
Phone: +46 (0)13-28 13 35
E-mail: danyo.danev@liu.se

Participants of the group

Name	Role	E-mail
Fawaizat Ahmed	Project Lead	fawah595@student.liu.se
Abdul Gaffar Abubakar Passum	Design Lead	abuab954@student.liu.se
Varun Gurupurandar	Test Lead	vargul25@student.liu.se
Mehran Mamivand	Documentation Lead	mehma172@student.liu.se

CONTENTS

1	introduction	1
2	Project Overview	1
2.1	Background	1
2.2	System Overview	1
2.3	Hardware	1
2.4	Software	2
2.5	Modules and Responsibilities	2
2.6	Verification Approach	2
2.7	Limitations	3
3	Plan for the project phases	3
4	Organization Plan	4
5	Document Plan	5
6	Development Method	5
7	Training Plan	5
8	Report Plan	7
9	Meeting Plan	7
10	Resource Plan	7
11	Milestones and Tollgates	8
11.1	Milestones	8
11.2	Tollgates	8
12	Activities	8
12.1	Other Activities	10
13	Time Plan	10
14	Plan for changes	11
15	Quality Plan	11
16	Risk Analysis	12
17	Project Closing	12

DOCUMENT HISTORY

Version	Date	Changes made	Sign	Reviewer
0.1	31-10-2025	Initial draft	AF, AG, VG, MM	DA
0.2	14-11-2025	2nd version	AF, AG, VG, MM	DA
1.0	12-01-2026	Final plan	AF, AG, VG, MM	DA

1 INTRODUCTION

The purpose of this Project Plan is to describe how Group 3 will execute the *Automatic Music Transcription System* (AMT) project within the TSKS23 CDIO course. The plan follows the LIPS project model and covers the overall project phases, organisation, documentation strategy, development method, resource usage, risk management, quality assurance, and the strategy for closing the project.

The project builds on the requirements defined and negotiated in the *Requirements Specification for the Automatic Music Transcription System*. That document describes what the final product shall do: take a piano signal recorded via microphone and/or obtained via MIDI, perform onset and offset detection, estimate pitches, and present the results as a human-readable list of notes (e.g. C4, D4, G4) with timing information. A MIDI file can optionally be used as a reference for evaluation.

This Project Plan complements those requirements by specifying *how* we intend to reach the desired result. It is a living document which has already been updated several times during the project as we learned more about the communication with the customer and supervisors, and it is also the reference for weekly status reports and tollgates.

2 PROJECT OVERVIEW

2.1 Background

The project develops an automatic music-transcription (AMT) prototype for piano, converting audio or MIDI input into symbolic notes with onsets, offsets and durations. The system is intended both as a demonstrator for the course and as a starting point for future student projects and research experiments in music signal processing.

The overall pipeline is designed to be feasible within the course time frame and aligned with the approved Requirements Specification (version 1.0). The focus is on monophonic and lightly polyphonic piano recordings, with the possibility to extend the system later towards more complex material and real-time scenarios.

2.2 System Overview

The system follows a four-stage pipeline (see the system design sketch):

1. **Input acquisition:** microphone recording of piano audio (.wav) or file upload (.wav, .mid as reference).
2. **Pre-processing:** resampling, normalisation, optional band-pass filtering; framing and STFT for a time-frequency view.
3. **Transcription engine:** onset detection based on an onset-strength envelope and peak picking, pitch detection with the YIN algorithm, offset detection and duration estimation, and mapping of fundamental frequencies to MIDI note numbers and note names.
4. **Output and UI:** a list of notes with onset time, offset, duration and note name; optional export to CSV and later MIDI; plots of waveform, spectrogram and detected onsets.

The baseline goal is to handle single notes and simple two-note chords in non-real time. If time permits, the system can be extended with better polyphonic handling and more interactive behaviour.

2.3 Hardware

The project uses the following hardware:

- Casio CTK-3500 keyboard (USB-B for MIDI).
- Streamplify MIC ARM (USB microphone).
- Laptops and CommSys lab computers (Intel i5 class, 8 GB RAM or more).
- A quiet lab room for recording.

2.4 Software

The implementation is based on Python 3.x and the following tools:

- `numpy`, `scipy` and `matplotlib` for numerical work and visualisation.
- `librosa` for onset–strength computation, YIN pitch detection and general audio handling.
- `mido` for parsing and handling MIDI files.
- `PyAudio` (or similar) for live recording.
- `PyQt5` for the graphical user interface.
- Cakewalk by BandLab for recording and exporting WAV and MIDI.

2.5 Modules and Responsibilities

The architecture is divided into five main modules that are reused across the LIPS documents:

- **Input Handling System (Module 1):** audio and MIDI input, file loading, resampling and normalisation.
- **Graphical User Interface (Module 2):** a PyQt–based interface for selecting input source, controlling recordings, and visualising waveforms, spectrograms and transcribed notes.
- **Backend / Core Processing Engine (Module 3):** onset/offset detection, YIN pitch estimation, and note mapping (frequency to MIDI and note names).
- **Database / Storage (Module 4):** file–based storage of recordings, transcriptions and meta–data (CSV/JSON and folder structure) with an interface that later can be replaced by SQL.
- **Output and Export (Module 5):** formatting the transcription as tables and export to CSV or text (and later MIDI).

The mapping between modules and internal roles is:

- M1 Input/Pre–processing – primarily Design Lead.
- M2 Onset/Offset – primarily Test Lead.
- M3 Pitch/Note Mapping – primarily Design Lead.
- M4 Integration and export – primarily Project Lead.
- M5 GUI & visualisation – primarily Documentation Lead.

2.6 Verification Approach

Verification is planned at both module and system level:

- **Unit tests** for onset detection, pitch estimation, note mapping, and input/output handling. These use small, controlled recordings (single notes, short chords) plus synthetic data.
- **System tests** where microphone recordings are aligned with reference MIDI and evaluated according to the performance requirements in the Requirements Specification: pitch accuracy, mean absolute error for onset (≤ 50 ms), offset (≤ 70 ms) and duration (≤ 0.1 s), as well as runtime for a 30 s clip (≤ 10 s on the target hardware).

The high–level architecture and signal pipeline are illustrated in the system design sketch and signal–flow diagrams referenced from this section.

The Automatic Music Transcription System (AMT), internally named *LiU MuSiCa*, aims to provide an educational and research–oriented tool for piano note transcription. The main focus is monophonic and lightly polyphonic piano pieces with relatively slow to moderate tempo, but the architecture is designed to be extendable towards more complex pieces and real–time operation.

The core functionality is:

- Record or load piano signals via microphone or WAV files.
- Optionally load a MIDI file as a reference signal.
- Detect onsets and offsets of notes using an onset–strength envelope and tuned peak picking.
- Estimate pitches using the YIN algorithm and map them to MIDI note numbers and note names (e.g. C#4, F5).
- Present a clear transcription with onset time, duration and note name.
- Export results and support further analysis (for example, comparing microphone recordings against ground–truth MIDI).

From the course description, the project is divided into mini–projects: single–pitch identification, transcription of simple pieces (no more than two notes simultaneously), and more complex and real–time scenarios. In practice, our project plan focuses on:

1. A robust and well–tested pipeline for single–pitch and simple two–note piano material.
2. A usable demonstrator with a GUI that can be shown to the customer and used in teaching.
3. A code base and documentation that can serve as a starting point for future student projects and research experiments.

The work is organised in the LIPS phases: *before* (planning and requirements), *during* (implementation and testing), and *after* (documentation, afterstudy, and handover).

2.7 Limitations

The following aspects are *not* part of the current project scope:

- Full polyphonic transcription of complex piano pieces.
- Real–time streaming with strict latency constraints.
- Support for instruments other than piano.
- Advanced music–theoretic analysis such as chord labelling, key detection or score engraving.

The system is therefore positioned as a research and teaching prototype for piano AMT, with a clear path for future extensions.

3 PLAN FOR THE PROJECT PHASES

The project is planned according to the three LIPS phases.

Before phase (Weeks 36–39)

- Study the project description and background on automatic music transcription.
- Assign internal roles (project manager, chief designer, documentation manager, test manager).
- Set up the technical environment: Python, `librosa`, `mido`, version control (GitLab) and Overleaf.
- Collect initial piano recordings and MIDI reference files.
- Produce and iterate the Requirements Specification, System Design Sketch and this Project Plan (Tollgate 2).

During phase (Weeks 40–50)

- Implement the core transcription pipeline: audio loading, onset/offset detection, YIN pitch detection, frequency-to-note mapping, and simple CLI tools (e.g. `test_transcribe.py`).
- Integrate support for both WAV and MIDI, including alignment of MIDI events for analysis and testing.
- Design and implement the GUI prototype with waveform and spectrogram visualisation, controls for loading/recording, parameter settings, and a transcription table.
- Extend the system with support for simple two-note chords.
- Produce and iterate the Design Specification and Test Plan (Tollgate 3).
- Conduct incremental module tests and system tests; collect test protocols for Tollgate 5.

After phase (Weeks 51–02)

- Final integration and bug fixing based on supervisor feedback.
- Prepare user manual, technical report, poster and web page.
- Perform afterstudy: evaluate time plan, risk handling and technical choices.
- Deliver final version of all documents for Tollgate 6 and present the demonstrator at the project conference.

4 ORGANIZATION PLAN

The project group consists of four members from the 6MDSI program. The Internal roles defined in the project description are assigned as follows:

- **Project Leader:** Ahmed Fawaizat Oyindamola (fawah595).
Responsible for overall planning, communication with the customer and supervisors, scheduling meetings, and ensuring that work follows this Project Plan.
- **Design Lead:** Abdul Gaffar Abubakar Passum (abuab954).
Responsible for the overall system architecture, module interfaces, and ensuring that the design meets the Requirements Specification.
- **Documentation Lead:** Mehran Mamivand (mehma172).
Responsible for LIPS documents, meeting minutes, and keeping the Git repository and Overleaf project organised.
- **Test Lead:** Varun Gurupurandar (vargu125).
Responsible for planning and supervising tests, writing the Test Plan and Test Protocols, and tracking bug reports.

The group works closely with the external roles:

- **Examiner and Customer:** Danyo Danev.
- **Supervisors:** David Nordlund (primary contact), Oksana Moryakova and Henrik Åkesson.
- **Consultants:** PhD students at the Division of Communication Systems.

For coordination, we use:

- the shared e-mail list `tsks23-group3@liu.se` for communication with the customer and supervisors,
- GitLab for version control of code and $\text{\LaTeX}$ sources,
- Overleaf for collaborative editing of LIPS documents, and
- a shared calendar for booking meetings and tollgates.

5 DOCUMENT PLAN

This section lists all documents to be produced during the project, their purpose, and the deadlines for the first draft. The documentation follows the LIPS/CDIO structure and is maintained in Overleaf with version control. Each document evolves through supervisor feedback until reaching Version 1.0 at its respective tollgate. Dependencies indicate which documents must exist before others can be completed.

6 DEVELOPMENT METHOD

The development method is iterative and incremental, compatible with the LIPS model. We divide the work into iterations of approximately one to two weeks. Each iteration includes:

- selecting a small set of requirements or bugs to focus on,
- implementing or refactoring code,
- writing or updating unit tests,
- updating documentation when needed, and
- demonstrating the progress to the group and, when relevant, to the supervisor.

From a technical perspective, the implementation is in Python using:

- `librosa`, `numpy` and `scipy` for signal processing,
- `mido` for parsing and handling MIDI files,
- `PyAudio` or similar libraries for live recording, and
- `PyQt` for the GUI.

For pitch detection, we use the YIN algorithm in `librosa`, with parameters tuned for a 61-key piano (roughly C2–C7). Onset and offset detection is based on `librosa`' onset-strength envelope and configurable peak-picking rules (`pre_max`, `post_max`, `delta`, `minimum gap`, and `minimum note duration`). Parameter settings are logged so that experiments can be reproduced.

Code is developed in feature branches and merged through GitLab with informal code reviews. The group follows a “working prototype first” approach: get a simple but functional pipeline running early (which has been achieved with `test_transcribe.py`) and then iteratively improve accuracy, robustness, and usability.

7 TRAINING PLAN

The project requires knowledge in several areas: music theory, digital signal processing, Python programming, and the LIPS project model. The training plan is therefore a mixture of self-study and practical exercises:

- **Music theory basics:** group members study the provided project description on scales, octaves and note naming, and watch introductory videos on piano notation and MIDI.
- **Signal processing and AMT:** review lecture notes and suggested papers on pitch detection, onset detection and automatic music transcription. Each member reads at least one research article and presents the main ideas at a group meeting.
- **Python and libraries:** short internal tutorials on using `librosa`, `mido` and plotting tools. The first weeks of coding (scripts for loading WAV/MIDI and plotting spectrograms) are treated as hands-on training.
- **LIPS documentation:** the documentation manager gives a short walkthrough of the LIPS templates and how to use Overleaf with the `lips.cls` class.

Training activities are scheduled primarily in the Before phase and early During phase, but self-study continues throughout the project whenever new techniques are needed.

Table 1: All project documents.

Document	Purpose (and Dependencies)	Deadline (First Draft)
Requirements Specification	Defines all functional, non-functional, and performance requirements. Forms the foundation for the Design Specification and Test Plan.	Week 37 (Completed)
System Design Sketch	High-level architecture description used during Tollgate 2. Depends on: Requirements Specification.	Week 38 (Completed)
Project Plan	Details project structure, organisation, time plan, milestones, tollgates, quality plan, document dependencies, and resource allocation. Depends on: System Design Sketch.	Week 38–39 (Completed)
Activity Plan (Excel)	Lists all project activities with dependencies, responsible members, planned hours, and weekly scheduling. Used to track progress toward tollgates. Depends on: Project Plan.	Week 45 (Completed)
Design Specification	Technical design of all modules (1–5), including audio I/O, GUI, backend engine, database schema, and output formats. Blueprint for implementation and testing. Depends on: Requirements Specification, System Design Sketch.	Week 49
Test Plan	Defines test methodology, test cases, module tests (1–5), success criteria, and requirement traceability. Depends on: Requirements Specification, Design Specification.	Week 49
Test Protocols (TG5 Required)	Step-by-step procedures for executing each test case. Supports system validation during implementation. Depends on: Test Plan.	Week 50
User Manual (TG5 Required)	Instructions for operating the AMT system (GUI usage, input methods, export features, troubleshooting). Depends on: Working prototype.	Week 51
Implementation / Prototype Documentation	Internal documentation of modules, algorithms, and integration. Used for maintainability and test verification. Depends on: Design Specification.	Week 50
Test Results Report	Summary of validation outcomes (accuracy, performance, robustness). Used for Tollgate 5 approval. Depends on: Test Protocols, Working Prototype.	Week 51
Final Technical Report (TG6)	Complete documentation of design, implementation, testing, and results, including analysis and reflections. Depends on: All previous documents.	January 9
Project Poster (TG6)	Visual summary of system design, methods, and results for final presentation. Depends on: Final Technical Report draft.	January 9
Project Web Page (TG6)	Online summary of project outcomes. Depends on: Poster + Final Technical Report.	January 9–16
Final Presentation Slides	Slides for oral presentation of the system, design, implementation, and results. Depends on: Poster + Test Results.	December 16
Afterstudy Report (TG6)	Reflection on project execution, teamwork, challenges, and improvements. Depends on: Project completion.	January 16

8 REPORT PLAN

We follow the reporting routines described in the project description: We follow the reporting routines described in the project description:

- **Weekly status reports** are sent every Monday to the supervisor and examiner. Each report contains:
 - a short summary of what has been done since the previous week,
 - encountered problems and how we plan to solve them,
 - whether the work followed the time plan, and
 - what is planned for the coming week.
- **Time reports** are maintained individually and collectively and attached to the weekly status report.
- **Milestone reports** (Tollgates 2, 3, 5 and 6) are accompanied by updated PDFs of the relevant LIPS documents.

Within the group, we use a shared document to track tasks, deadlines and responsibilities. Short internal check-in meetings are held at least once per week to complement the written reports.

9 MEETING PLAN

The meeting structure is as follows:

- **Internal group meetings:** at least once per week, normally in the CommSys lab. The agenda includes status updates, planning for the next week, and discussion of technical issues. Minutes are kept in a simple template and stored in the Git repository.
- **Supervisor meetings:** approximately every second week, or when needed around tollgates. At least one meeting is held ahead of each tollgate to confirm that the deliverables are on track.
- **Customer meetings / Tollgates:** Tollgates 2, 3, 5 and 6 are formal meetings with the customer and examiner where the corresponding documents and demonstrations are presented.
- **Ad-hoc consultations:** shorter meetings or e-mails with PhD consultants for specific technical questions (for example choice of onset detector parameters or evaluation metrics for AMT).

The project leader is responsible for booking meetings, preparing agendas, and ensuring that minutes are written and stored.

10 RESOURCE PLAN

The main resources available to the project are:

- **Human resources:** four students, each expected to contribute about 240 hours over the project period.
- **Hardware:**
 - Casio CTK-3500 keyboard.
 - MIC ARM by Streamplify (USB microphone).
 - Cables: USB-C to USB-A, USB-C to USB-C, USB-A to USB-B.
 - Laptops and the computers in the CommSys lab.
- **Software:**
 - Python with `librosa`, `numpy`, `scipy`, `mido`, `PyAudio` and `PyQt`.
 - Cakewalk by BandLab for recording and exporting WAV and MIDI.

- GitLab for version control and issue tracking.
- Overleaf for $\text{\LaTeX}$ documents.
- **Supervision:** approximately 15 hours of supervision and 25 hours of expert consultancy available from the division.

Resource usage is tracked in the time reports. Any need for additional resources (extra equipment, extended supervision) will be discussed with the customer as early as possible.

11 MILESTONES AND TOLLGATES

11.1 Milestones

Table 2: The milestones for the project.

Nr	Description	Week
1	Repository setup, environment configuration, and successful audio I/O pipeline	46
2	First working onset detector and stable offset estimation on simple piano tones	47
3	First working pitch detector (YIN) with correct note mapping on monophonic signals	48
4	GUI prototype with waveform and spectrogram visualisation, and full pipeline integration (Input $\rightarrow$ Processing $\rightarrow$ Output)	48–49
5	Integrated demonstrator capable of transcribing multiple audio files, with history storage and export features	50–51
6	Full system ready for supervisor review and documentation freeze for Tollgate 3	50

11.2 Tollgates

Table 3: The tollgates for the project.

Nr	Description	Week
1	Approval of project idea and initial Requirements Specification	35–36 (Completed)
2	Requirements Specification (v1.0), System Design Sketch, and Project Plan delivered	37–38 (Completed)
3	Design Specification and Test Plan (first drafts), submitted early enough for supervisor review before holidays	49–50
5	Fully working demonstrator, Test Protocols, and User Manual delivered; covers implementation and testing phase	51–1
6	Technical Report, Poster, Afterstudy, and Project Web Page. First drafts due 9 Jan, final versions due 16 Jan	2–3

12 ACTIVITIES

This section contains all activities that are to be performed during the Automatic Music Transcription System project. Each activity has a clear purpose, estimated workload, and defined dependencies. The numbering below is used for planning, tracking, and references in the time plan.

Nr	Activity	Description	Est. Time	Dep.
1	Repository & environment setup	Set up shared Git repository, folder structure, coding conventions and basic Python environment. Provides a stable base for all implementation activities.	20 h	–
2	Audio I/O pipeline	Implement recording from microphone and loading of .wav files, plus saving test audio. Verify that audio can be captured and read reliably for further processing.	40 h	A1
3	Onset detection baseline	Implement an initial onset detector (e.g. spectral flux + thresholding) and visualise detected onsets to verify behaviour on simple piano recordings.	60 h	A2
4	Pitch detection baseline	Implement fundamental frequency estimation using YIN and/or autocorrelation methods. Test on isolated notes to ensure stable f_0 estimates.	60 h	A2
5	Note mapping & duration estimation	Map estimated f_0 values to piano notes, estimate note onset/offset and duration, and ensure consistency with the Requirements Specification (tolerance limits).	60 h	A3, A4
6	Integration of transcription pipeline	Integrate A3–A5 into a complete pipeline that takes audio as input and outputs a list of notes (onset, offset, pitch, duration) in a structured format (e.g. CSV).	60 h	A3–A5
7	GUI prototype	Develop a simple graphical user interface that allows the user to load or record audio, run the transcription pipeline (A6), and display the resulting list of notes.	50 h	A2, A6
8	Results storage & history	Implement storage of transcription results (CSV/MIDI files and/or lightweight database or file-based history) and basic history/browsing functionality.	30 h	A6
9	Testing framework	Design and implement tests for core modules (onset, pitch, mapping, integration), prepare reference data, and implement scripts to compute accuracy and timing metrics.	50 h	A3, A4, A6
10	Performance evaluation	Evaluate the system against the performance requirements (onset/offset accuracy, duration error, and runtime constraints) using the testing framework and documented test cases.	40 h	A9
11	GUI polish & integration	Refine the GUI layout, integrate history/export (A8), improve usability and error handling, and ensure consistency with the implemented pipeline.	40 h	A7, A8
12	Documentation & design updates	Update the Design Specification, Project Plan, user-facing documentation and figures to reflect the implemented solution. Prepare material needed for final reporting and tollgates.	60 h	A1–A11
13	Final demonstration & delivery preparation	Prepare and rehearse the final demo, poster and presentation. Package final code, documentation and results according to course requirements.	40 h	A11, A12

12.1 Other Activities

This subsection contains activities not directly related to the technical implementation but necessary for the project.

Nr	Activity	Description	Est. Time
14	Weekly internal meetings and communication	Stand-ups, planning, follow-up and communication with supervisors (David, Henrik, Danyo).	15 h
15	Administration and reporting	Administration, reporting, and time logging in the shared Excel sheet.	10 h

Finally, the table below summarises the main phases again in ActivityList form, focusing on higher-level project activities. (Optional: can be removed if the examiner prefers a single activity list.)

13 TIME PLAN

The time plan is based on the activities described in the previous section and on the official course schedule. The project runs from early September to mid-January. A simplified week-by-week plan is:

- **Weeks 36–37:** Activities A1–A2, initial meetings with customer and supervisors.
- **Weeks 38–39:** Early requirements work and initial system design sketch.
- **Weeks 40–43:** Activities A3–A4; prototypes for onset and pitch detection; first CLI transcription script.
- **Weeks 44–46:** Activities A5–A6; integrated transcription engine and evaluation on simple test material.
- **Weeks 47–49:** Activities A7–A8; GUI prototype, connection to backend, and initial demonstration.
- **Weeks 50–51:** Activities A9–A10; Test Plan, systematic experiments and parameter tuning.
- **Weeks 52–01:** Activity A11–A12; documentation updates and preparations for Tollgates 3 and 5.
- **Weeks 02–03:** Activity A13 and Tollgate 6 deliverables; final polishing, afterstudy, poster and web page.

This plan is updated in the weekly status reports when delays or changes occur. The main constraint is that all documents and the demonstrator must be ready by mid-January, as communicated by the supervisor. Time is logged in the shared Excel sheet per Professor Danyo's instruction.

A more detailed view linking calendar weeks to activities A1–A13 is:

Week 45 (3–9 Nov)	Freeze RS v0.3; submit to David. Complete System Design Sketch draft. Draft Time Plan.
Week 46 (10–16 Nov)	A1 Repository and environment setup. A2 Audio I/O pipeline implemented and verified. Start A3 Onset detection baseline.
Week 47 (17–23 Nov)	Continue A3 Onset detection. A4 Pitch detection implementation. Start A5 Note mapping and duration estimation.
Week 48 (24–30 Nov)	A5 duration estimation finalised. A6 Integration of transcription modules (onset, pitch, mapping, duration). A7 GUI prototype development.
Week 49 (1–7 Dec)	A8 Results storage and history. A9 Testing framework. GUI refinements and intermediate integration testing.
Week 50 (8–14 Dec)	A10 Performance evaluation using the test framework. A11 GUI polish and integration. A12 Documentation and design updates.
Week 51 (15–21 Dec)	A13 Final demo preparation: poster, presentation, code packaging, final submission.

Follow-up and Revisions

Weekly internal meetings are used to review progress, reassign tasks and update the Excel time plan. Supervisor follow-ups are planned as needed (target: once every 1–2 weeks). The time plan is revised dynamically to reflect completed work and any schedule changes agreed with the supervisors.

Roles & Load Distribution

- **Project Lead (Ahmed):** integration, coordination, weekly reports, meetings with supervisors.
- **Design Lead (Abdul):** M1/M3 pipelines and architecture diagrams.
- **Test Lead (Varun):** M2 onset/offset methods, test datasets and metric scripts.
- **Documentation Lead (Mehran):** GUI stub, figures and document maintenance.

14 PLAN FOR CHANGES

Changes in requirements, scope or time plan are inevitable in a project of this size. Our strategy for handling changes is:

- All change proposals (for example new functionality, removal of features or modifications of algorithms) are first discussed at an internal meeting.
- If the change influences the Requirements Specification or any deliverable related to a tollgate, the project leader contacts the customer and supervisor to agree on the modification.
- Accepted changes are documented in the relevant LIPS documents and in the weekly status report. When appropriate, the version history tables are updated.
- The time plan and Activity List are adjusted if the change affects effort or deadlines. Critical changes are highlighted at the next supervisor meeting.

Typical examples of changes that may occur include: narrowing the scope from complex real-time polyphonic transcription to robust monophonic/simple polyphonic transcription; modifying the GUI features; or changing the evaluation data set.

15 QUALITY PLAN

Quality assurance is important both for the technical product and for the documentation.

For the software we apply the following measures:

- **Code standards:** use clear naming, comments and modular functions. Shared helper classes (e.g. `AudioProcessor`, `OnsetOffsetDetector`, `EnhancedPitchDetector`) are documented in docstrings.
- **Version control:** all code and documents are tracked in GitLab. No changes are made directly on the main branch; instead feature branches and merge requests are used.
- **Testing:** the Test Plan defines unit tests (small scripts that verify onset and pitch detection on controlled examples) and system tests (end-to-end transcription of recorded MIDI/WAV pairs). Test results are logged in Test Protocols.
- **Peer review:** before important milestones, each group member reviews the others' code and documentation sections.

For the documentation we ensure that:

- all LIPS templates are followed and the documents are internally consistent,
- each document has a maintained version history,
- spelling and formatting are checked before tollgates, and
- feedback from the customer and supervisors is incorporated promptly.

16 RISK ANALYSIS

The main project risks and mitigation strategies are summarised below.

R1: Underestimation of implementation effort.

The algorithms for onset and pitch detection and their integration can require more time than initially planned.

Mitigation: focus first on a working pipeline for simple monophonic material; postpone advanced features (e.g. complex polyphony, real-time streaming) if time becomes limited.

R2: Poor transcription accuracy.

The chosen methods and parameter settings may not give sufficient accuracy for the customer's expectations.

Mitigation: perform early experiments on several data sets, including BandLab loops and keyboard recordings; schedule time for systematic tuning of YIN and onset parameters; consult experts if the performance is too low.

R3: Integration problems between GUI and backend.

The GUI may lag or block when running heavy signal processing.

Mitigation: keep a clear separation between GUI and backend logic; test the GUI early with a small dataset; if necessary, move heavy processing to background threads or use progress indicators.

R4: Absence or overload of group members.

Illness, exam periods or external work may temporarily reduce available capacity.

Mitigation: cross-train group members so that more than one person can work on each module; keep the Activity List updated; re-distribute tasks as needed.

R5: Missed tollgate deadlines.

Delays in documentation or implementation could push tollgates too late.

Mitigation: treat the internal deadlines suggested by the supervisor (e.g. drafts before holidays, final versions by mid-January) as hard constraints; send early drafts for feedback instead of waiting for a "perfect" version.

17 PROJECT CLOSING

The project is considered complete when:

- the AMT demonstrator can transcribe the defined set of test pieces (single notes and simple piano pieces) with acceptable accuracy,
- all LIPS documents are finalised and approved at Tollgate 6,
- the user manual and technical report are delivered, and
- the poster and web page are published and presented at the project conference.

Before closing the project we will:

- clean the GitLab repository (remove temporary branches and obsolete files, tag the final release),
- ensure that all code can be run following the instructions in the technical report,
- archive meeting minutes, test protocols and time reports, and
- conduct an internal afterstudy meeting where each group member reflects on what worked well and what can be improved in future projects.

The final afterstudy document will be submitted together with the other Tollgate 6 deliverables.